LAPORAN UTS

MACHINE LEARNING

SmartReceipt Analyzer

Klasifikasi Kategori Item Struk Menggunakan
TF-IDF Character N-Gram dan Multinomial Naive Bayes

Disusun untuk memenuhi tugas Ujian Tengah Semester

Mata Kuliah [Machine Learning] **Proyek** [Project Based Learning (PBL)] **Tim** [SmartReceipt Analyzer] **Tahun** [2026]

Daftar Isi

1. Pendahuluan 3
2. Deskripsi Data 3
3. Preprocess 4
4. Extract Feature 4
5. Training Strategy 5
6. Train Method 5
7. Hasil Evaluasi Performa 6
   7.1. Evaluasi Utama pada Data Uji 6
   7.2. Evaluasi Integrasi Model Ekspor 6
   7.3. Evaluasi Eksperimen Notebook 7
   7.4. Katalog Produk Indonesia (Penambahan Data Sintetis) 7
   7.5. Keterbatasan dan Tindak Lanjut 7
8. Kesimpulan 7

1. Pendahuluan

SmartReceipt Analyzer merupakan aplikasi pencatatan pengeluaran yang membaca item pada struk belanja melalui *Optical Character Recognition* (OCR). Komponen machine learning pada proyek ini bertugas mengklasifikasikan teks nama item hasil OCR ke dalam kategori pengeluaran secara otomatis. Prediksi dilakukan secara lokal (*on-device*) di aplikasi Flutter sehingga tidak membutuhkan API atau koneksi internet.

Alur Sistem

Teks item hasil OCR → Preprocessing → Ekstraksi TF-IDF → Multinomial Naive Bayes → Kategori / **Perlu Review**

2. Deskripsi Data

Dataset yang digunakan adalah `Dataset_SmartReceipt_Sintetis.csv`. Data dibuat secara sintetis dari daftar produk dan variasi penulisan yang lazim muncul pada struk maupun keluaran OCR. Dataset ini mendukung pembuatan prototipe, namun belum dapat dianggap sebagai representasi seluruh struk asli di Indonesia.

Karakteristik	Nilai
Total baris	6.026
Produk/kata kunci unik	1.419
Jumlah kategori	15
Data sintetis produk spesifik	5.212 baris
Data kata kunci generik	814 baris
Data latih	4.474 baris
Data validasi	762 baris
Data uji	790 baris

Kolom utama pada dataset dijelaskan sebagai berikut.

Kolom	Deskripsi
`id`	Identitas setiap sampel.
`product_id`	Identitas produk atau kata kunci.
`nama_produk`	Nama produk dalam bentuk normal.
`receipt_text`	Teks produk yang menyerupai keluaran OCR pada struk; digunakan sebagai fitur masukan model.
`kategori`	Label target yang diprediksi model.
`gaya`	Variasi teks, misalnya `clean`, `abbr`, `trunc`, `nospace`, `vowel`, `dropword`, dan `kode_plu`.
`grup`	Merek atau kelompok produk untuk evaluasi GroupKFold.
`split`	Pembagian data: `train`, `val`, atau `test`.
`sumber`	Asal contoh: `sintetis` atau `kata_kunci`.

Lima belas kategori target adalah Bayi & Anak, Elektronik, Hewan Peliharaan, Hiburan & Hobi, Kesehatan, Lainnya, Makanan, Minuman, Pakaian & Fashion, Pendidikan & ATK, Perawatan Diri, Rokok & Tembakau, Rumah Tangga, Tagihan & Utilitas, dan Transportasi.

Pembagian dilakukan berdasarkan produk, bukan hanya berdasarkan baris. Artinya, satu produk hanya berada pada satu *split*, sehingga produk dalam data uji tidak pernah muncul di data latih. Data kata kunci generik hanya dimasukkan ke data latih agar model memahami istilah umum tanpa membocorkan sampel ke validasi atau pengujian.

3. Preprocess

Preprocessing bertujuan mengurangi *noise* yang tidak membawa informasi kategori, terutama akibat format struk dan kesalahan OCR. Tahapan pada notebook pelatihan adalah:

1. Mengubah seluruh teks menjadi huruf kecil.
2. Menghapus angka, nominal, barcode/PLU, ukuran, dan satuan seperti `600ML`, `1KG`, atau `50000`.
3. Menghapus simbol dan karakter selain huruf.
4. Merapikan spasi menjadi satu spasi.
5. Menghapus sampel dengan hasil preprocessing kosong.

Contoh hasil preprocessing:

Teks Awal	Hasil
`8992761134019 INDOMIE GRG 85G`	`indomie grg`
`AQUA BTL 600ML`	`aqua btl`
`TOKEN LISTRIK 50000`	`token listrik`

Pada aplikasi Flutter, `IndonesianTextPreprocessor` memperluas preprocessing dengan normalisasi Unicode, penghapusan mata uang dan persentase, penanganan karakter berulang akibat OCR, tokenisasi, penghapusan *stopword* dan satuan, serta ekspansi singkatan atau merek. Contohnya, `indomie` menjadi `mie instan`, `aqua` menjadi `air mineral`, dan `panadol` menjadi `obat`.

Untuk meningkatkan ketahanan terhadap kesalahan OCR, data latih diaugmentasi dengan enam salinan ber-*noise*. *Noise* dibuat melalui penggantian karakter mirip, penghapusan karakter, penggandaan karakter, dan pertukaran posisi karakter pada tingkat 4% hingga 15%. Augmentasi hanya dilakukan pada data latih agar evaluasi tetap adil.

4. Extract Feature

Fitur diekstraksi menggunakan `TfidfVectorizer` dari scikit-learn.

Parameter	Nilai
Analyzer	`char_wb`
Rentang n-gram	2 sampai 4 karakter
Pembobotan	TF-IDF dengan `sublinear_tf=True`
Normalisasi vektor	L2
Maksimum fitur saat pelatihan	20.000
Vocabulary model ekspor	20.000 fitur

Pendekatan *character n-gram* dipilih karena teks item struk sering tidak lengkap atau mengalami salah baca OCR. Contohnya, `INDOMIE GORENG` dapat muncul sebagai `INDOMIE GRG`, `INDME GORENG`, atau

`INDOMIEGORENG`. N-gram karakter mampu mempertahankan kemiripan pola karakter pada variasi tersebut, sedangkan pendekatan berbasis kata penuh lebih mudah kehilangan kecocokan.

Skema `char_wb` membentuk n-gram dalam batas kata dengan menambahkan spasi di awal dan akhir kata. Frekuensi istilah dihitung dengan `1 + ln(tf)`, dikalikan nilai IDF, kemudian dinormalisasi menggunakan norma L2. Vektor inilah yang menjadi masukan model klasifikasi.

5. Training Strategy

Strategi pelatihan dirancang agar model mampu menghadapi produk baru dan teks dengan *noise* OCR.

1. Data dibagi menjadi *train*, *validation*, dan *test* berdasarkan produk.
2. Data latih diaugmentasi dengan enam versi *noise OCR*.
3. Data validasi digabung dengan satu versi *noise OCR* untuk memilih hiperparameter tanpa menggunakan data uji.
4. Model dibandingkan pada `ngram_range (2,4), (2,5), (3,5)`; parameter smoothing `alpha 0,01`, `0,03`, `0,1`, `0,3`; dan opsi `fit_prior` benar/salah.
5. Model terbaik dipilih menggunakan **macro-F1** pada data validasi agar setiap kategori berbobot setara.
6. Model diuji pada teks bersih dan teks dengan *noise OCR* dari produk yang belum pernah dilihat.
7. Evaluasi tambahan menggunakan 5-fold GroupKFold berdasarkan `grup` atau merek. Seluruh produk dari satu kelompok dapat disembunyikan dari data latih sehingga pengujian lebih ketat.
8. Setelah evaluasi, model final dilatih ulang menggunakan gabungan train dan validation. Produk pada data test tidak pernah disertakan dalam model ekspor sehingga metrik tetap dapat diverifikasi ulang secara jujur. Guard anti-kebocoran juga diterapkan pada data tambahan: baris katalog yang produknya tumpang tindih dengan data validasi atau uji dibuang dari data latih (41 baris dibuang dari data uji, 4 baris dari data validasi).

Model menerapkan mekanisme penolakan prediksi. Ambang keyakinan dipilih pada data validasi dengan target akurasi prediksi otomatis minimal 90%. Model ekspor menggunakan ambang **0,90**. Jika probabilitas tertinggi di bawah nilai itu, keluaran adalah **Perlu Review** agar pengguna dapat mengoreksi kategori secara manual.

Koreksi pengguna disimpan sebagai pasangan teks item ternormalisasi dan kategori koreksi. Data ini dapat diekspor sebagai dataset tambahan untuk proses pelatihan berikutnya.

6. Train Method

Metode klasifikasi yang digunakan adalah **Multinomial Naive Bayes**. Metode ini sesuai untuk fitur non-negatif berdimensi tinggi seperti bobot TF-IDF. Skor setiap kelas dihitung dengan:

$$\text{skor}_{\text{kelas}} = \log \text{prior}_{\text{kelas}} + \sum \left(\text{TF-IDF}_{\text{fitur}} \times \log \text{probabilitas}_{\text{fitur} \mid \text{kelas}} \right)$$

Skor seluruh kelas diubah menjadi probabilitas posterior melalui softmax. Kategori dengan probabilitas tertinggi dipilih jika melewati ambang keyakinan; jika tidak, sistem memberikan label **Perlu Review**.

Alasan pemilihan metode ini adalah sebagai berikut.

1. Cepat dilatih dan ringan saat inferensi di perangkat mobile.
2. Cocok untuk klasifikasi teks pendek dengan jumlah fitur besar.
3. Bekerja baik bersama TF-IDF dan n-gram karakter.
4. Parameter dapat diekspor ke JSON dan dihitung ulang di Dart tanpa layanan eksternal.

5. Menghasilkan probabilitas yang dapat digunakan untuk menentukan item yang perlu diperiksa pengguna.

Implementasi Flutter mereplikasi tahapan pelatihan: ekstraksi `char_wb` n-gram, TF sublinear, pembobotan IDF, normalisasi L2, penghitungan skor Naive Bayes, dan softmax. Oleh sebab itu, hasil inferensi aplikasi konsisten dengan model Python yang diekspor.

7. Hasil Evaluasi Performa

7.1. Evaluasi Utama pada Data Uji

Model dievaluasi pada 790 data uji dari `Dataset_SmartReceipt_Sintetis.csv`. Produk pada data uji tidak pernah digunakan pada data latih. Evaluasi dilakukan dalam dua skenario: teks bersih dan teks yang diberi simulasi *noise OCR*.

Skenario	Jumlah	Akurasi	Macro-F1
Teks bersih (model awal)	790	70,6%	0,669
Teks + noise OCR (model awal)	790	67,5%	0,637
Teks bersih (model selaras Flutter)	790	72,3%	0,687
Teks bersih (model v2 + katalog 170 produk)	790	74,9%	0,712
Teks + noise OCR (model v2)	790	68,2%	0,646

Model selaras Flutter dilatih ulang dengan aturan preprocessing yang sama dengan aplikasi. Model v2 memasukkan tambahan 170 produk Indonesia dari katalog lintas 15 kategori (`katalog_produk_indonesia_v1.json`), menghasilkan total 1.190 baris augmentasi tambahan. Dibanding model awal, akurasi teks bersih naik 4,3 poin (70,6% → 74,9%) dan macro-F1 naik 0,043 (0,669 → 0,712). Pada skenario *noise OCR*, model v2 memperoleh 68,2% akurasi dan 0,646 macro-F1, sedikit lebih baik daripada model awal (67,5% dan 0,637). Sebagai pembanding, akurasi pada produk yang sudah dikenal dengan *noise OCR* baru adalah **96,0%** (model awal: 96,6%). Selisih terhadap data uji produk baru menunjukkan tantangan utama model, yaitu melakukan generalisasi kategori untuk nama produk yang belum pernah dilihat. Metrik di atas dihitung dengan guard anti-kebocoran sehingga produk yang tumpang tindih dengan data validasi/uji tidak pernah ada di data latih; pengujian struk nyata tetap perlu dilakukan.

7.2. Evaluasi Integrasi Model Ekspor

Selain evaluasi utama, `eval_dataset_dummy.json` digunakan untuk menguji integrasi model JSON pada aplikasi Flutter. File ini berisi 72 sampel berlabel dari 15 kategori. Model JSON diuji melalui implementasi Dart yang sama dengan aplikasi.

```
flutter test test/model_evaluator_test.dart --reporter expanded
```

Metrik	Hasil
Jumlah sampel	72
Akurasi	0,931 atau 93,1%
Macro-F1	0,894
Weighted-F1	0,955
Status pengujian	8 test lulus

Hasil ini membuktikan model JSON dapat dimuat dan dieksekusi dengan benar di Flutter, sedangkan metrik diperoleh dari prediksi aktual model, bukan nilai statis. Evaluasi 72 sampel ini adalah pengujian integrasi aplikasi, bukan metrik performa utama model.

7.3. Evaluasi Eksperimen Notebook

Notebook menyediakan evaluasi terpisah untuk data uji bersih, data uji dengan *noise OCR*, laporan klasifikasi per kelas, confusion matrix, dan rata-rata 5-fold GroupKFold per merek. Hasil utama untuk 790 data uji di atas diperoleh dari eksekusi notebook pada Google Colab. Evaluasi *noise OCR* model v2 dijalankan dengan `evaluate_noise.py`, yang mereplikasi metode notebook: noise diterapkan pada teks mentah sebelum preprocessing Flutter, dengan seed 8 untuk data uji dan seed 9 untuk sampel produk yang sudah dikenal. Metrik per kelas, confusion matrix, dan GroupKFold dapat ditambahkan setelah output lengkap tersedia.

7.4. Katalog Produk Indonesia (Penambahan Data Sintetis)

Dataset awal berisi 1.419 produk. Untuk memperluas cakupan, ditambahkan `katalog_produk_indonesia_v1.json` yang memuat 170 produk dan layanan umum Indonesia tambahan, tersebar merata di seluruh 15 kategori. Setiap produk menghasilkan 7 variasi teks struk (bersih, kapital, ukuran, alias, tanpa spasi, terpotong, kode PLU) sehingga total 1.190 baris augmentasi. Dataset ini bersifat sintetis dan ditandai `sumber=katalog_sintetis` untuk membedakannya dari data asli.

7.5. Keterbatasan dan Tindak Lanjut

Kombinasi preprocessing selaras Flutter dan penambahan 170 produk katalog menghasilkan model v2 dengan akurasi teks bersih 74,9% dan macro-F1 0,712 pada 790 data test sintetis; dengan *noise OCR* model memperoleh 68,2% dan 0,646. Semua metrik dihitung dengan guard anti-kebocoran produk. Walaupun demikian, dataset utama masih sintetis sehingga hasil belum menggambarkan sepenuhnya kondisi struk nyata. Evaluasi lanjutan yang diperlukan adalah mengumpulkan minimal 100 item dari 30–50 struk asli, menjalankan OCR aplikasi, memberi label kategori secara manual, lalu menghitung akurasi, precision, recall, macro-F1, weighted-F1, dan confusion matrix.

Data struk nyata tidak boleh digunakan untuk pelatihan sebelum evaluasi selesai. Setelah itu, data nyata dan koreksi pengguna dapat digunakan untuk melatih ulang model secara berkala sehingga cakupan produk, merek, dan variasi kesalahan OCR meningkat.

8. Kesimpulan

SmartReceipt Analyzer menggunakan TF-IDF berbasis character n-gram dan Multinomial Naive Bayes untuk mengklasifikasikan item struk ke 15 kategori pengeluaran. Kombinasi preprocessing, augmentasi noise OCR, pemisahan data berdasarkan produk, dan ambang keyakinan membantu model menghadapi teks struk yang tidak konsisten.

Pengujian utama model awal pada 790 data test menghasilkan akurasi 70,6% dan macro-F1 0,669 untuk teks bersih, serta akurasi 67,5% dan macro-F1 0,637 untuk teks dengan noise OCR. Setelah preprocessing training dan Flutter diselaraskan, model v2 dengan penambahan 170 produk katalog mencapai akurasi 74,9% dan macro-F1 0,712 pada teks bersih serta 68,2% dan 0,646 pada teks dengan noise OCR, dengan guard anti-kebocoran produk. Pengujian integrasi pada 72 sampel dummy menghasilkan akurasi 93,1%, macro-F1 0,894, dan weighted-F1 0,955. Pengujian pada struk asli tetap diperlukan agar performa model dapat dinilai secara realistis sebelum digunakan secara luas.